



Génie Logiciel

CER | Restauration

Vendredi 30 janvier 2026

TABLE DES MATIÈRES

Étude de cas : Burnout	2
AAV	2
Mots clés	3
Contexte	3
Problématique	3
Contraintes	3
Généralisation	3
Livrables	4
Pistes de solutions	4
Plan d'actions	4
Démarche : Burnout	5
Définition: Mots clés	5
Workshop	10
Qu'est-ce qu'un diagramme UML ?	10
Diagrammes UML structurels	11
Diagrammes de classe	12
Diagrammes de séquençage	13
Diagrammes de cas d'utilisation	14
Diagrammes d'activité	15
Diagrammes d'objets	16
Pourquoi utiliser l'UML en POO ?	16
Analyse et résolution : Burnout	17
Conclusion	17
Capitalisation	18
Ressources	18

ÉTUDE DE CAS : BURNOUT

AAV

- 1 Citer l'utilité de MVC et MVVM et leurs différences
- 1 Décrire le rôle de chaque composant MVC et MVVM
- 1 Connaître la syntaxe du langage C#
- 3 Pratiquer le langage C#
- 3 Pratiquer les architectures MVC et MVVM

MOTS CLÉS

- Modularité
- MVC
- MVVM
- .NET6
- C#
- ASP.NET core
- Séparation des préoccupations
- Principe SOLID
- Framework modernes

CONTEXTE

2F Roaming souhaite refondre l'architecture de son application afin de réduire les coûts de développement. De plus, la format° des équipes est nécessaire.

PROBLÉMATIQUE

Comment faciliter l'architecture de l'app de l'entreprise afin de réduire les coûts ?

CONTRAINTES

- Framework imposé
- Coûts de dev
- Formation équipe

GÉNÉRALISATION

- POO
- UML
- Framework moderne
- Communication technique

LIVRABLES

- Nouvelle archi
- Plan de formation

PISTES DE SOLUTIONS

- Principes du SOLID
- Architecture MVC ou MVVM
- Diagramme UML
- Séparation des couches

PLAN D'ACTIONS

- Etudier les principes du SOLID
- Etudier les diagrammes UML (Séquence, Classe, Cas d'utilisation)
- Etudier les types d'architecture (MVC, MVVM)
- Proposer une nouvelle architecture
 - Réalisation des diagrammes UML
 - Explication et plan
- Documentation des nouvelles pratiques

DÉMARCHE : BURNOUT

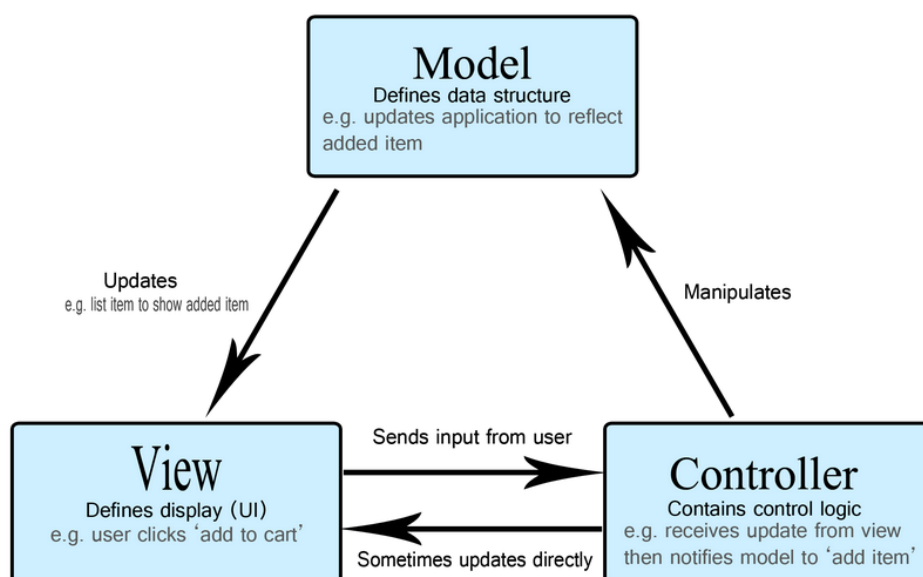
DÉFINITION : MOTS CLÉS

Modularité: En informatique et en programmation, la modularité consiste à diviser un système en modules ou composants distincts. Chaque module gère une fonctionnalité spécifique et fonctionne de manière indépendante. Elle simplifie la conception, le développement, les essais et la maintenance en permettant de se concentrer sur une partie à la fois sans affecter le reste du système.

MVC (Modèle-Vue-Contrôleur): MVC (Modèle-Vue-Contrôleur) est un modèle de conception logicielle couramment utilisé pour implémenter les interfaces utilisateurs, les données et la logique de contrôle. Il met l'accent sur la séparation entre la logique métier du logiciel et l'affichage. Cette « séparation des responsabilités » permet une meilleure répartition du travail et une maintenance facilitée. D'autres modèles de conception sont basés sur MVC, comme MVVM (Modèle-Vue-ViewModèle), MVP (Modèle-Vue-Présentateur) et MVW (Modèle-Vue-Whatever).

Les trois parties du modèle de conception logicielle MVC peuvent être décrites ainsi :

- **Modèle** : Gère les données et la logique métier.
- **Vue** : Gère la présentation et l'affichage.
- **Contrôleur** : Transmet les commandes aux parties modèle et vue.



Le Modèle Le modèle définit quelles données l'application doit contenir. Si l'état de ces données change, le modèle notifie généralement la vue (pour que l'affichage soit mis à jour si besoin) et parfois le contrôleur (si une logique différente est nécessaire pour contrôler la vue mise à jour). Dans notre application de liste de courses, le modèle spécifierait quelles données doivent être présentes dans chaque élément de la liste — article, prix, etc. — et quels éléments sont déjà présents.

La Vue La vue définit comment les données de l'application doivent être affichées. Dans notre application de liste de courses, la vue définirait comment la liste est présentée à l'utilisateur et recevrait les données à afficher depuis le modèle.

Le Contrôleur Le contrôleur contient la logique qui met à jour le modèle et/ou la vue en réponse aux actions des utilisateur·ice·s de l'application. Par exemple, notre liste de courses pourrait comporter des formulaires et des boutons permettant d'ajouter ou de supprimer des articles. Ces actions nécessitent la mise à jour du modèle, donc la saisie est envoyée au contrôleur, qui manipule alors le modèle comme il convient, puis envoie les données mises à jour à la vue.

C#: C# est un langage de programmation orientée objet, fortement typé, dérivé de C et de C++, ressemblant au langage Java. Il est utilisé pour développer des applications web, ainsi que des applications de bureau, des services web, des commandes, des widgets ou des bibliothèques de classes. En C#, une application est un lot de classes où une des classes comporte une méthode Main, comme cela se fait en Java.

C# est destiné à développer sur la plateforme .NET, une pile technologique créée par Microsoft pour succéder à COM.

Les exécutables en C# sont subdivisés en assemblies, en namespaces, en classes et en membres de classe. Un assembly est la forme compilée, qui peut être un programme (un exécutable) ou une bibliothèque de classes (une dll). Un assembly contient le code exécutable en CIL, ainsi que les symboles. Le code CIL est traduit en langage machine au moment de l'exécution par la fonction just-in-time de la plateforme .NET.

ASP.NET Core: ASP.NET Core est une infrastructure open source multiplateforme et hautes performances pour la création d'applications web modernes à l'aide de .NET. L'infrastructure est conçue pour le développement d'applications à grande échelle et peut gérer n'importe quelle charge de travail de taille, ce qui en fait un choix robuste pour les applications de niveau entreprise.

Pourquoi utiliser ASP.NET Core ?

- **Framework unifié** : ASP.NET Core est un framework web complet et entièrement intégré avec des composants intégrés prêts pour la production pour gérer tous vos besoins de développement web.
- **Productivité complète de la pile** : créez plus d'applications plus rapidement en permettant à votre équipe de travailler sur une pile complète, du front-end au serveur principal, à l'aide d'une seule infrastructure de développement.

- **Sécurisé par conception** : ASP.NET Core est conçu avec une sécurité qui constitue une préoccupation majeure et inclut la prise en charge intégrée de l'authentification, de l'autorisation et de la protection des données.
 - **Prêt pour le cloud** : que vous déployiez sur vos propres centres de données ou dans le cloud, ASP.NET Core simplifie le déploiement, la supervision et la configuration.
 - **Performances et scalabilité** : gérez les charges de travail les plus exigeantes grâce aux performances de pointe d'ASP.NET Core dans le secteur.
 - **Fiable et mature** : ASP.NET Core est utilisé et prouvé à hyperscale par certains des plus grands services du monde, notamment Bing, Xbox, Microsoft 365 et Azure.
-

Séparation des préoccupations: La séparation des préoccupations (SoC) est un principe de conception logiciel qui consiste à diviser un programme informatique en sections distinctes, chacune traitant d'une préoccupation spécifique ou d'un aspect particulier du programme. L'objectif principal de la SoC est de réduire la complexité du code, d'améliorer la maintenabilité et de faciliter la réutilisation des composants.

Principe SOLID: Les principes SOLID consistent en cinq pratiques et directives pour un code propre, maintenable et flexible lors de la programmation orientée objet (OOP).

- **Single Responsibility Principle** ou Principe de Responsabilité Unique : une classe ne doit avoir qu'une seule responsabilité (et non une seule tâche) et une seule raison de changer.
- **Open Closed Principle** ou Principe d'Ouverture/Fermeture : les classes doivent être ouvertes aux extensions et fermées aux modifications.
- **Liskov Substitution Principle** ou Principe de Substitution de Liskov : les sous-classes doivent pouvoir hériter et implémenter toutes les méthodes et propriétés de la superclasse.
- **Interface Segregation Principle** ou Principe de Ségrégation des interfaces : les interfaces ne doivent pas contenir plus de méthodes qu'il n'en faut pour les classes qui les implémentent.
- **Dependency Inversion Principle** ou Principe d'Inversion de Dépendance : les classes ne doivent pas dépendre d'autres classes, mais d'interfaces ou de classes abstraites.

Les avantages de l'application des SOLID Principes permettent que le code soit :

- **Clair, propre et beau** : les logiciels et les codes sont plus faciles à comprendre, plus compréhensibles, plus efficaces et tout simplement plus beaux.
 - **Facile à entretenir** : grâce à une structure claire et concise, il est plus facile d'entretenir et de maintenir le code, qu'il soit nouveau ou qu'il ait évolué au fil du temps, même si plusieurs personnes sont impliquées.
 - **Adaptable, extensible, réutilisable** : grâce à une meilleure lisibilité, à une réduction des complexités et des responsabilités ainsi qu'à une réduction des dépendances entre les classes, le code peut être plus facilement édité, adapté, étendu par des interfaces et réutilisé de manière flexible.
 - **Moins sujet aux erreurs** : avec un code propre et une structure simple, les modifications apportées à une partie n'ont pas d'impact involontaire sur d'autres domaines ou fonctions.
 - **Plus sûr et plus fiable** : en réduisant ou en éliminant les points faibles, les incompatibilités ou les erreurs, on améliore la fonctionnalité et la fiabilité des systèmes, et donc la sécurité.
-

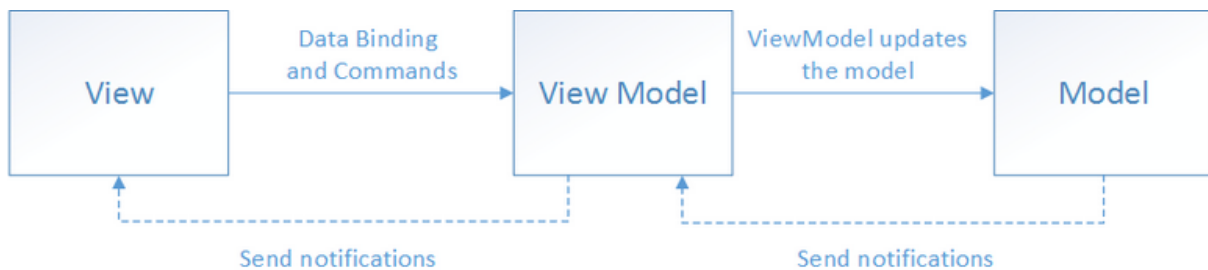
Framework modernes: Un framework est un ensemble de composants logiciels réutilisables qui permettent de développer de nouvelles applications plus efficacement. Il fournit une structure de base et des fonctionnalités prêtes à l'emploi, ce qui permet aux développeurs de se concentrer sur la logique métier spécifique de leur application plutôt que de réinventer la roue à chaque fois.

- **Améliorer la qualité du code** : Les frameworks logiciels contiennent des composants logiciels conçus selon des normes de programmation élevées. Les développeurs peuvent utiliser le framework logiciel avec la certitude que moins de bogues affecteront les codes sous-jacents
- **Gagner du temps sur le développement** : Les frameworks logiciels améliorent l'efficacité de la programmation et les entreprises peuvent les utiliser pour publier des applications fonctionnelles plus rapidement. Avec un bon framework logiciel, les développeurs peuvent se concentrer sur l'écriture de code de haut niveau qui gère la logique métier plutôt que sur des modules de codage de base.
- **Meilleure sécurité logicielle** : Avec une base de code élargie, il est difficile pour les développeurs de détecter les problèmes de sécurité du code et d'y répondre. En revanche, un bon framework logiciel comprend des points de contrôle de sécurité prêts à l'emploi qui permettent aux développeurs de renforcer plus facilement la sécurité du code et des données.
- **Révision efficace du code** : En utilisant un framework, les équipes logicielles peuvent valider leurs applications à l'aide de scénarios de test complets et d'une couverture de code. De plus, les développeurs trouvent qu'il est plus facile de déboguer et de corriger les problèmes de code dans un framework bien structuré.

- **Flexibilité de développement** : Ils peuvent retenir le code spécifique au projet tout en échangeant différents frameworks adaptés à leurs objectifs. Cela réduit les réécritures de code que les développeurs doivent effectuer.

MVVM: Le modèle MVVM (Modèle-Vues-Vues-Modèle) permet de séparer de manière nette la logique métier et la logique de présentation d'une application de son interface utilisateur (IU). Le maintien d'une séparation nette entre la logique d'application et l'IU permet de résoudre de nombreux problèmes de développement et de faciliter le test, la maintenance ainsi que l'évolution d'une application. Il permet également d'améliorer considérablement les opportunités de réutilisation du code.

Il existe trois composants principaux dans le modèle MVVM : le modèle, la vue et le modèle de vue. Chacun sert un objectif distinct. Le diagramme ci-dessous montre les relations entre les trois composants

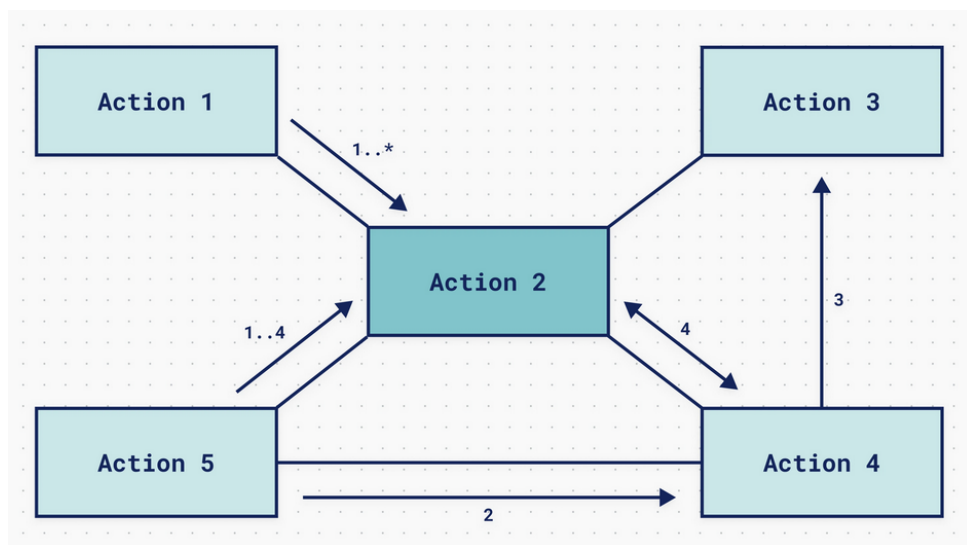


.NET6: .NET 6 est une plateforme de développement open source, multiplateforme et hautes performances créée par Microsoft. Elle permet aux développeurs de créer une variété d'applications, notamment des applications web, des applications de bureau, des services cloud, des applications mobiles et des applications IoT (Internet des objets).

WORKSHOP

QU'EST-CE QU'UN DIAGRAMME UML ?

Les diagrammes UML sont la représentation de la notation UML. Elle se compose d'éléments standardisés (tels que des composants, des classes et des cas d'utilisation) qui sont connectés à l'aide de relations (flèches) pour représenter l'architecture, l'implémentation et la structure d'un système logiciel par ailleurs complexe.



Les diagrammes UML représentent le plus souvent les éléments suivants :

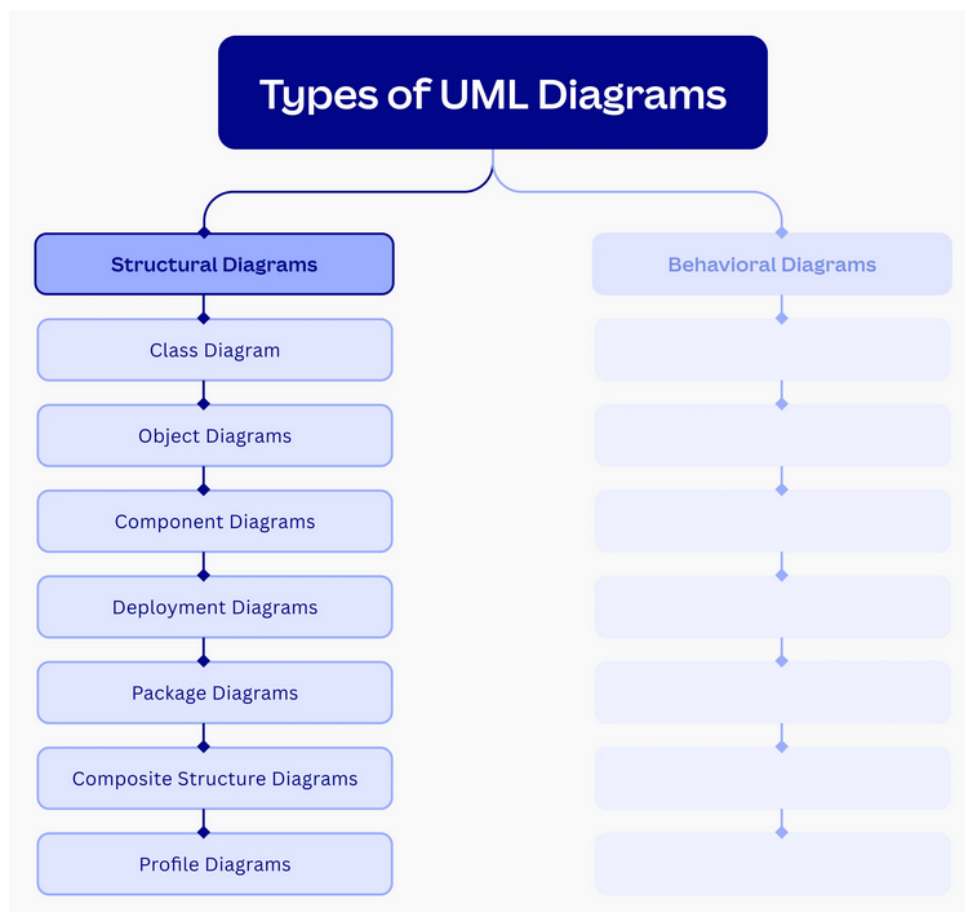
- Les composants du système et comment ils interagissent avec les autres
- L'interface utilisateur du système
- Les activités ou travaux effectués par les composants ou les interfaces
- Comment différentes entités interagissent avec les composants et les interfaces
- La manière dont le système fonctionne

Le modèle UML est composé d'une combinaison de diagrammes. Les 14 types de diagrammes UML représentent le modèle de manière structurelle ou comportementale, comme vous pouvez le voir ci-dessous.

DIAGRAMMES UML STRUCTURELS

Les diagrammes structurels UML représentent la structure statique d'un système. Ces diagrammes utilisent des objets, des opérations, des relations et des attributs pour analyser ou décrire cette structure. Sous les diagrammes UML structurels se trouvent les types suivants :

- Diagramme de classes
- Diagramme de composants
- Diagramme de déploiement
- Diagramme d'objets
- Diagramme de packages
- Diagramme de structure composite
- Diagramme de profils



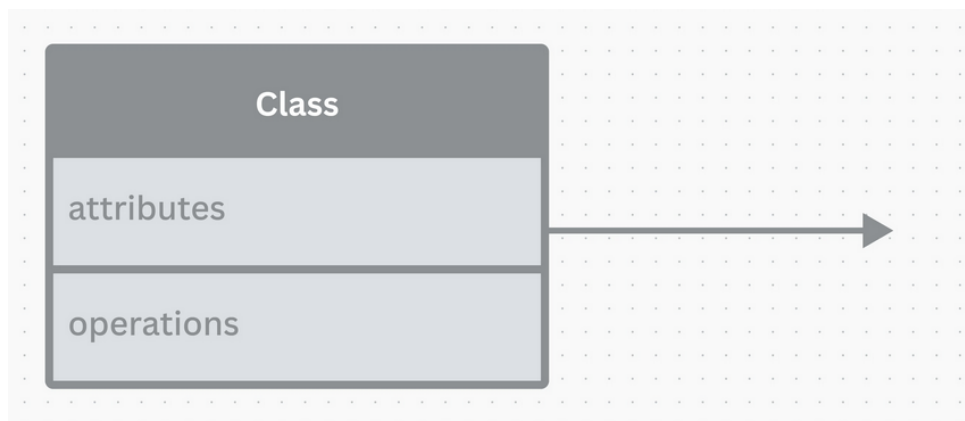
DIAGRAMMES DE CLASSE

Un diagramme de classe UML est généralement utilisé pour visualiser les composants d'un système dans la programmation orientée objet. Il représente les classes (avec leur visibilité, leurs portées, leurs signaux, leurs types de données, leurs packages, leurs interfaces et leurs objets) et leurs relations. Pour une vue plus détaillée, ces classes peuvent également être divisées en sous-classes.

Généralement utilisés pour la modélisation conceptuelle d'une application, les diagrammes de classe UML peuvent également être facilement traduits en code de programmation et sont aussi utilisés dans la modélisation de données.

Par principe, les diagrammes de classe UML sont représentés à l'aide des éléments suivants :

- **Classes** : ce composant est constitué d'un rectangle où le nom de la classe (sa première lettre en majuscule) est récrit en gras et au milieu. Les attributs (valeurs qui détaillent une instance de classe) sont alignés à gauche avec les premières lettres en minuscules. Pour finir, les opérations, ou les actions que la classe peut exécuter, sont alignées à gauche avec les premières lettres en minuscules.
- **Relations entre classes** : représentées par un trait en pointillés ou plein selon le type de relation (association, héritage, réalisation, dépendance, agrégation ou composition).



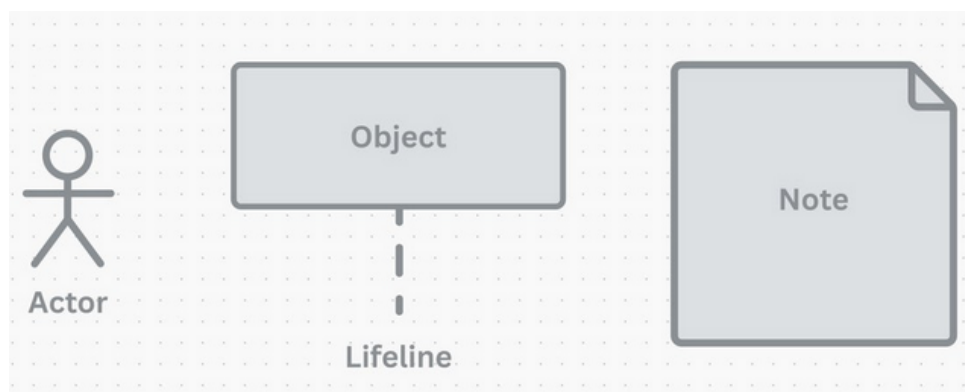
DIAGRAMMES DE SÉQUENÇAGE

Un diagramme de séquence UML affiche la séquence requise pour exécuter une fonctionnalité. Il représente les processus et les objets impliqués dans la fonctionnalité et la séquence de messages échangés entre eux.

Les diagrammes de séquence, ou diagrammes d'événements, sont généralement utilisés en génie logiciel pour capturer la manière dont les opérations sont effectuées et la manière dont le codage logiciel peut être effectué pour réaliser la séquence de cas d'utilisation.

Les diagrammes de séquence UML sont représentés comme suit :

- **Acteur** : un bonhomme bâton.
- **Participant individuel à l'interaction** : une ligne pointillée avec un rectangle en haut.
- **Temps entre le début et la fin de l'interaction** : représenté par un rectangle avec une bordure fine sur la ligne de vie.
- **Message entre deux lignes de vie** : texte à côté d'une flèche indiquant la direction du message.
- **Remarques sur les éléments** : annotations.



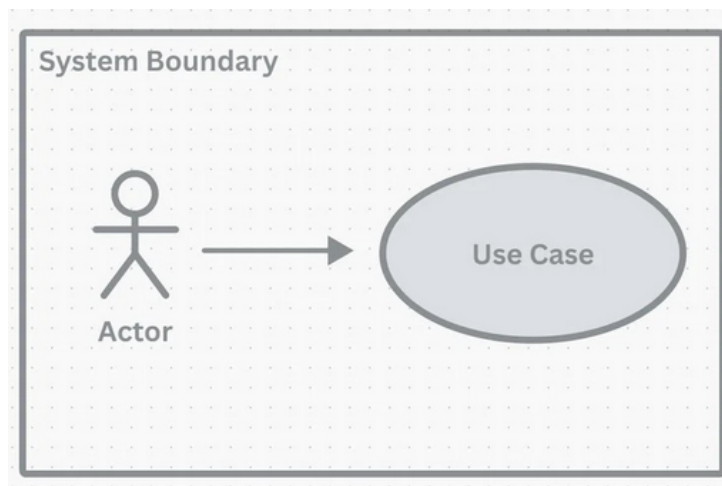
DIAGRAMMES DE CAS D'UTILISATION

Un diagramme de cas d'utilisation UML, comme son nom l'indique, décrit le cas d'utilisation d'un système. Autrement dit, il affiche les interactions possibles qu'un utilisateur peut avoir avec un système et permet de présenter le système du point de vue de l'utilisateur final.

Les diagrammes de cas d'utilisation sont utilisés dans les pratiques de développement de logiciels, d'entreprise ou de modélisation commerciale pour offrir aux parties prenantes non techniques une vue d'ensemble du système. Ils permettent de comprendre la manière dont le système sera conçu.

Les diagrammes de cas d'utilisation UML sont représentés comme suit :

- **Acteur** : le participant qui interagit avec le système, représenté sous la forme d'un bonhomme bâton.
- **Cas d'utilisation** : une fonction système automatisée ou manuelle décrite par un verbe et représentée par une ellipse. Les cas d'utilisation doivent être associés à des acteurs.
- **Lien de communication** : utilisé pour montrer l'association entre les acteurs et les cas d'utilisation. Il est représenté par des flèches.
- **Limite du système** : représenté sous forme de rectangles. Les acteurs et les cas d'utilisation entrent dans ce cadre.



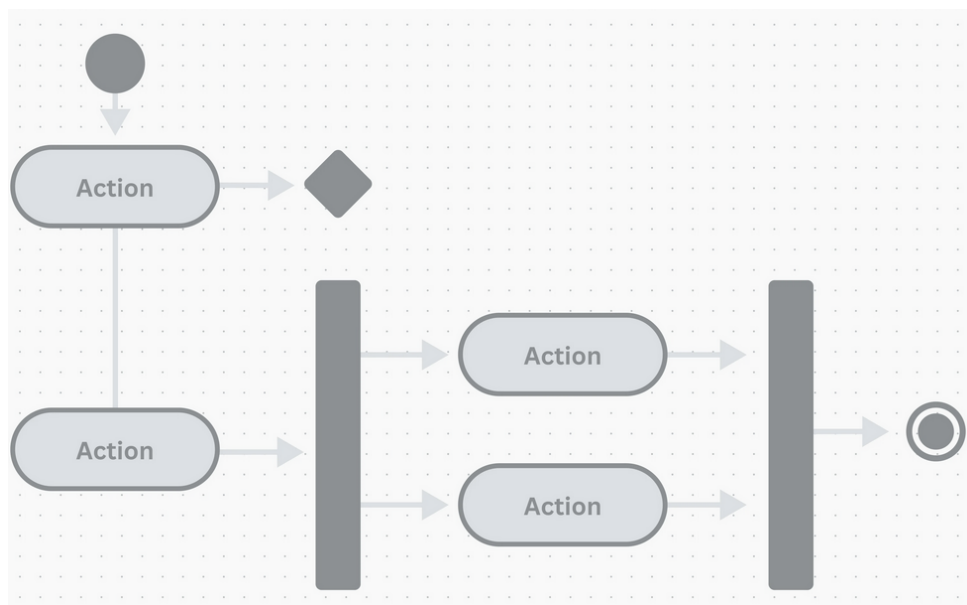
DIAGRAMMES D'ACTIVITÉ

Un diagramme d'activité UML décrit les activités, les actions et les flux de travail des processus informatiques ou organisationnels. Il s'agit essentiellement d'un logigramme avancé qui modélise le flux des opérations d'un système ou d'une organisation.

Les diagrammes d'activité sont généralement utilisés dans le design de systèmes embarqués pour visualiser la relation entre les cas d'utilisation et les processus métier.

Les diagrammes d'activités UML sont représentés comme suit :

- **Actions** : rectangles arrondis
- **Décisions** : losanges
- **Activités simultanées** : barres
- **Début du processus** : cercle noir
- **Arrêt du processus** : cercle noir avec bordure

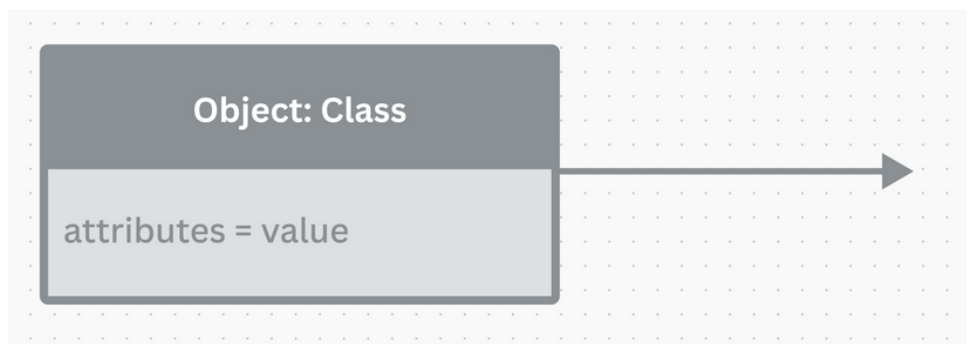


DIAGRAMMES D'OBJETS

Un diagramme d'objet UML est une instance d'un diagramme de classe où des exemples réels sont utilisés pour représenter le système. Il représente les objets et les relations entre eux, ce qui peut être utile pour créer des diagrammes de classe ou pour vérifier l'exactitude et l'exhaustivité d'un diagramme. Les diagrammes d'objet sont également largement utilisés dans la programmation orientée objet.

Tout comme les diagrammes de classe, les diagrammes d'objets sont également représentés comme suit :

- **Objet** : un rectangle avec le nom de l'objet, séparé par deux points de sa classe et souligné, et les attributs de l'objet. Contrairement aux classes, les attributs d'objet doivent se voir attribuer des valeurs.
- **Relations entre objets** : des lignes qui dénotent une association, un héritage, une réalisation, une dépendance, une agrégation ou une composition.



POURQUOI UTILISER L'UML EN POO ?

En POO, on manipule des concepts abstraits (classes, héritage, interfaces). Sans schéma, on se perd vite dans la complexité. L'UML permet de :

- Communiquer entre développeurs (et avec les clients).
- Anticiper les erreurs de conception avant d'écrire la moindre ligne de code.
- Générer automatiquement du squelette de code.

ANALYSE ET RÉOLUTION : BURNOUT

CONCLUSION

CAPITALISATION

AAV	Commentaire	Statut
Citer l'utilité de MVC et MVVM et leurs différences	Les concepts de MVC et MVVM sont expliqués, différences et utilités bien identifiées, exemples et schémas fournis.	Acquis
Décrire le rôle de chaque composant MVC et MVVM	Rôles du Modèle, Vue, Contrôleur et Modèle de Vue explicités, avec illustrations et explications claires.	Acquis
Connaître la syntaxe du langage C#	Principes de base, structure, types, classes et méthodes C# présentés avec exemples.	Acquis
Pratiquer le langage C#	Exercices et exemples pratiques de code C# réalisés, manipulation des concepts fondamentaux.	Acquis
Pratiquer les architectures MVC et MVVM	Mise en œuvre concrète des architectures MVC et MVVM dans des exemples ou mini-projets.	Acquis

RESSOURCES

- **Documentation officielle de Git** : <https://git-scm.com/doc>
- **Pro Git Book** : <https://git-scm.com/book/en/v2>
- **GitHub Learning Lab** : <https://lab.github.com/>
- **POO** : <https://docs.microsoft.com/fr-fr/dotnet/csharp/fundamentals/object-oriented/>
- **C# Guide** : <https://docs.microsoft.com/fr-fr/dotnet/csharp/>
- **LINQ** : <https://docs.microsoft.com/fr-fr/dotnet/csharp/programming-guide/concepts/linq/>
- **Async/Await** : <https://docs.microsoft.com/fr-fr/dotnet/csharp/programming-guide/concepts/async/>